

Taming Spooky Actions at a Distance: A Discipline of Quantum Software Testing

Mohammad Reza Mousavi

mohammad.mousavi@kcl.ac.uk

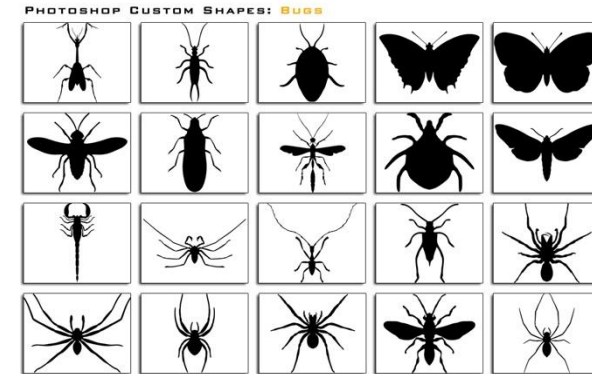
University of Salerno
03 December 2025



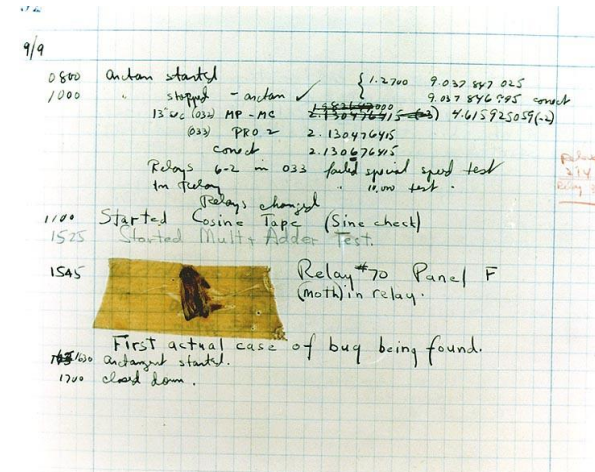
King's Quantum



Testing Quantum Systems



Taxonomy of Hybrid Quantum-Classical Bugs



Debugging Quantum Systems



King's Quantum

Vision:

King's Quantum will:

BUILD a community on the *transformative application* of quantum science,
BROADEN collaborations across Faculties and industry, and
to **BOOST** interdisciplinary research funding.

BUILD

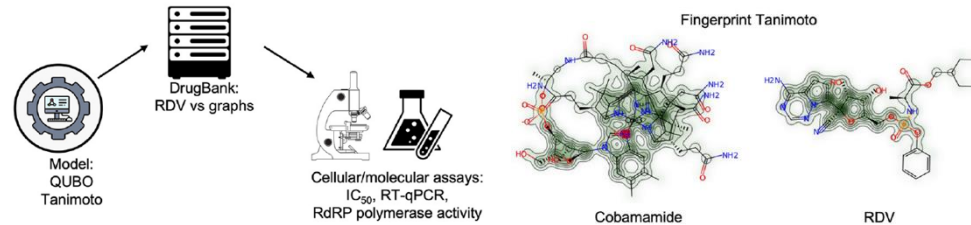
BROADEN

BOOST

Rationale – UK's National Quantum Strategy 2023

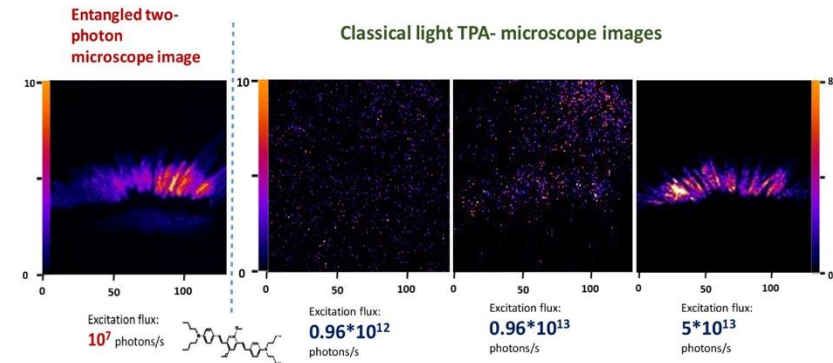
“Drive the adoption and use of quantum technologies in the UK to deliver benefits for the economy and society, as well as our national security”

Quantum algorithms for antiviral searches (Immunology)



Jimenez-Guardeno *et al.* (KCL) *PLOS Comput Biol* **18**, 1010330 (2022)

Quantum enhanced bio-imaging (Randall Centre for Cell & Molecular Biophysics)



Varnavski & Goodson *J. Am. Chem. Soc.* **142**, 12966 (2020)

Quantum magnetoencephalography (Neuroimaging)



Rea *et al.*, *Ann NY Acad Sci*, **1517** 107 (2022)

Quantum geodesy / gravimetry for environmental monitoring (Geography)



The transportable optical clock of the PTB (left) is housed inside a trailer (right). Credit: PTB Braunschweig, [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)

Computing

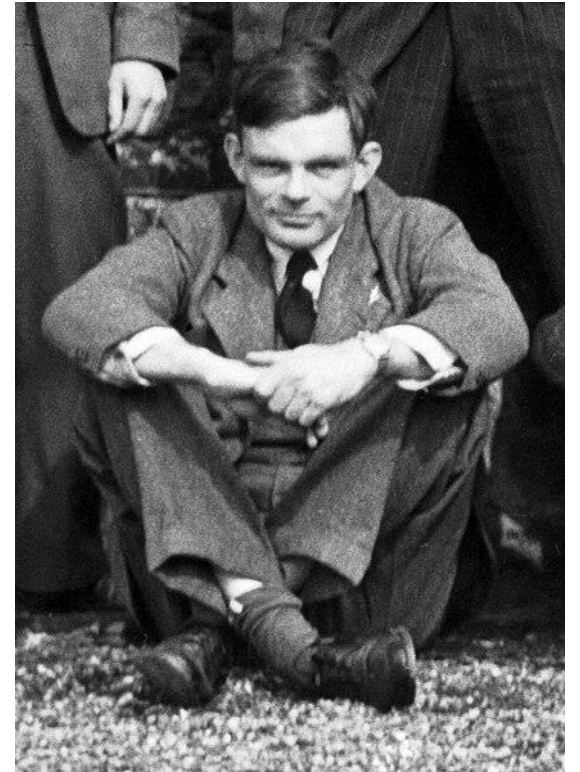
Calculating using **algorithms** working on **bits**

Example:

Prime decomposition: for a positive integer n ,
find prime number **factors** = $[f_0, f_1, \dots, f_n]$, such that
 $n = f_0 \dots f_n$

An algorithm $\text{factors}(n)$:

- for each number f from 2 to $\sqrt{n}$;
 - if n **divisible by** f ; return **append(factors(f), factors (n/f))**
 - if $n = 1$ or n **not divisible by any** f , return $[n]$



Alan Turing (1912-1954)

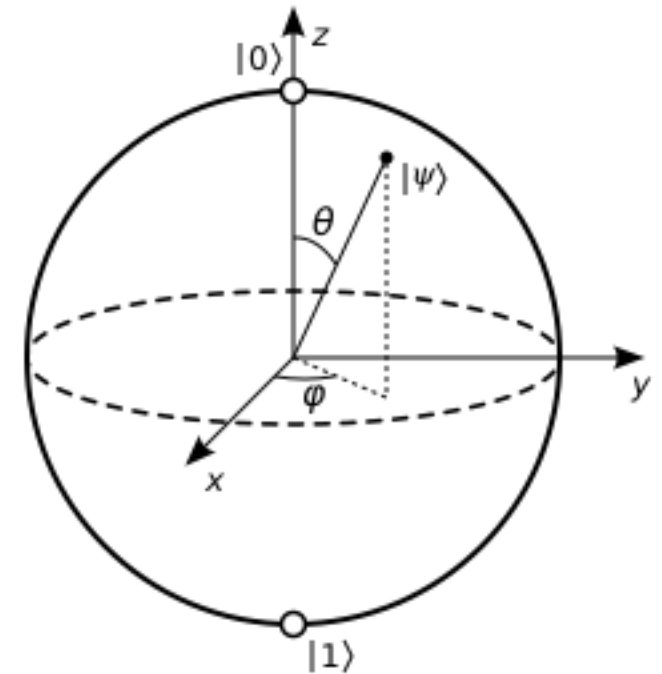
Photo: Wikimedia Commons

Quantum Computing

Calculating using **algorithms** working on **qubits**

Bits vs. Qubits:

- **bits** are either **0 or 1**;
qubits can be in a **superposition of states**
(**superposition**)
- operations on bits are **local**;
operations on qubits, can affect **others**
(**entanglement**: spooky action at a distance)
- bits can be **inspected** and **copied**;
inspecting a qubit will **collapse its state**; they **cannot be copied**



Qubit in Bloch Sphere

Photo: WikiMedia Commons

Quantum Computing

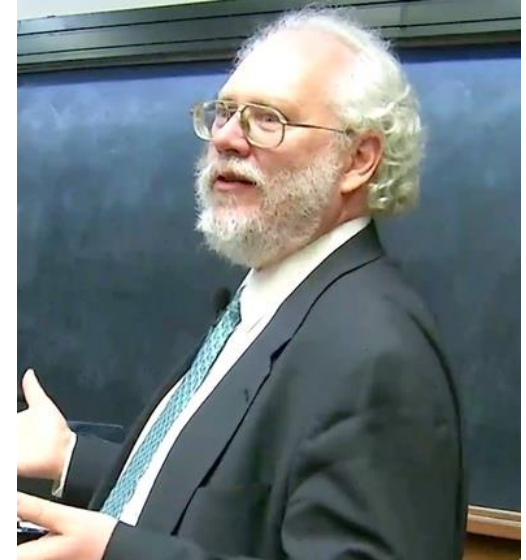
Calculating using **algorithms** working on **qubits**

Example:

Prime decomposition: for a positive integer n ,
find prime number **factors** = $[f_0, f_1, \dots, f_n]$, such that
 $n = f_0 \dots f_n$

An algorithm $q\text{-factors}(n)$:

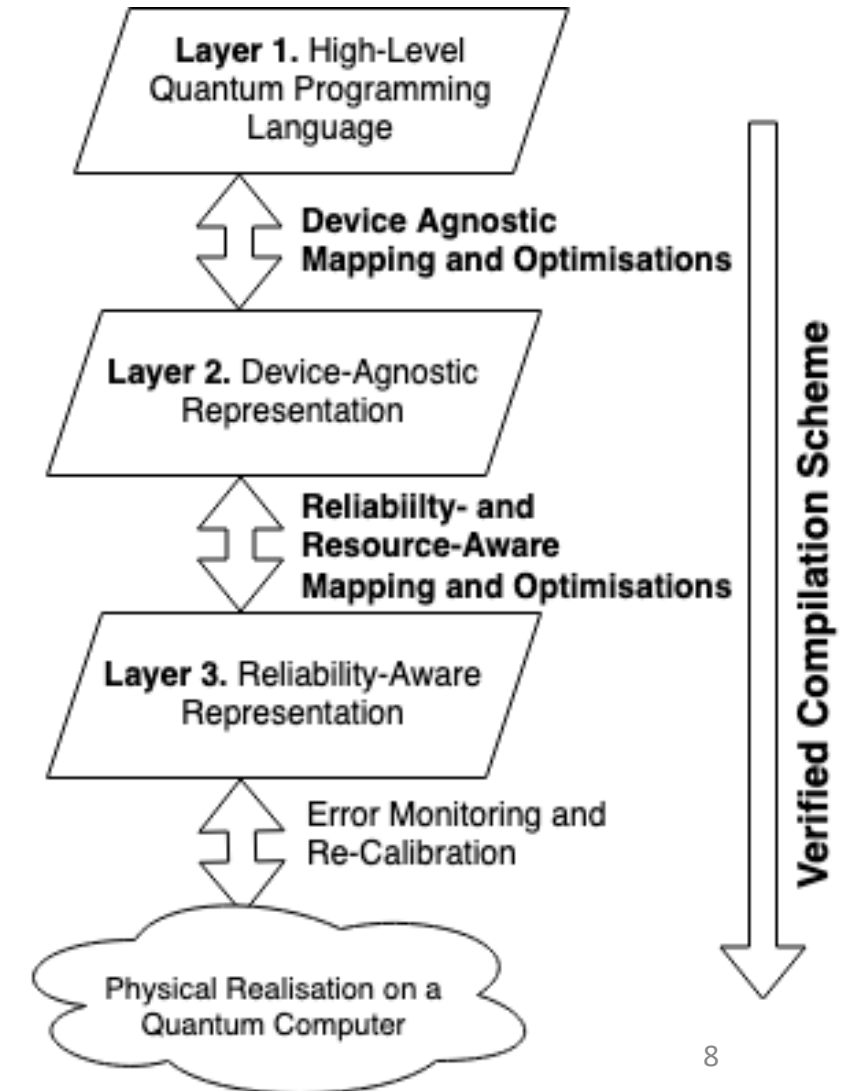
- Guess a **random number r** ;
- if n and r have a **common divisor gcd** ,
 - append (**$q\text{-factor}(n / gcd)$** , $q\text{-factor}(gcd)$)
- else, **improve the guess to r'** ,
so that n and r' do have a common divisor and repeat



Peter Shor (1959 -)

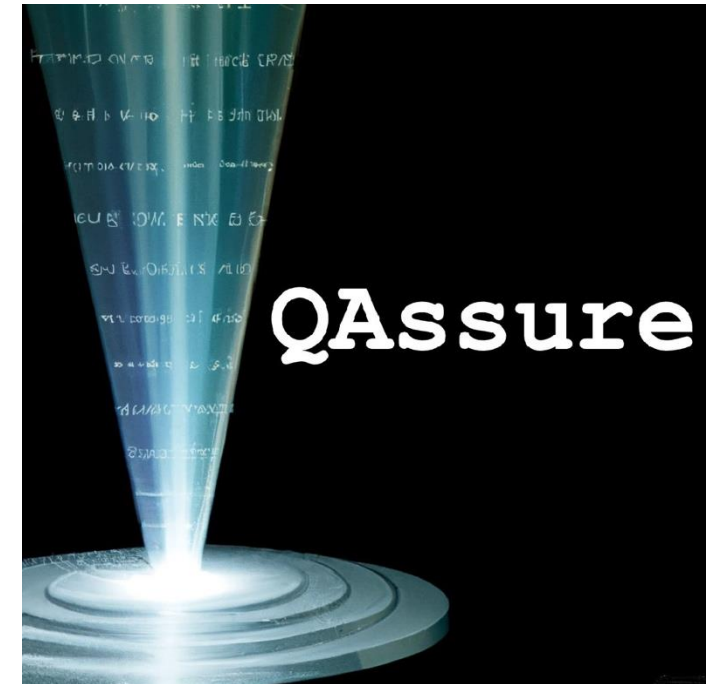
King's Quantum – Verified Simulation of Large-Scale Quantum Systems

- Developing a compositional and expressive method for **specifying quantum systems** and defining **transformations** among them.
- Checking **timed and stochastic conformance** among mappings
- **Evaluating** the developed methodology on large-scale case studies for simulating **fermionic systems**



QAssure: Accelerating Assurance for Quantum Secure Communications

- Security assurance framework for quantum-secure communications, primary focus:
Quantum Key Distribution (QKD)
- 11 partners from large and small enterprises, universities, and the National Physical Laboratory
- InnovateUK funding, as part of the Quantum Challenge within the National Quantum Technologies Programme

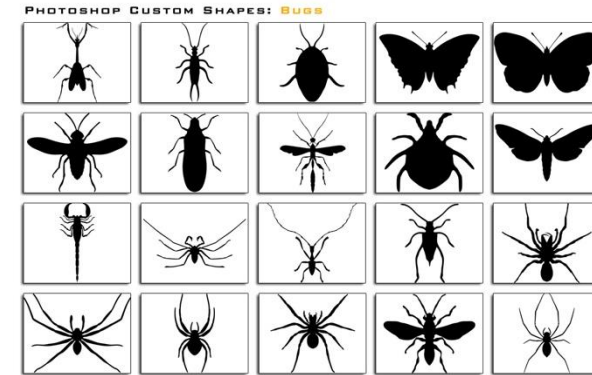




King's
Quantum ✓



Testing
Quantum Systems

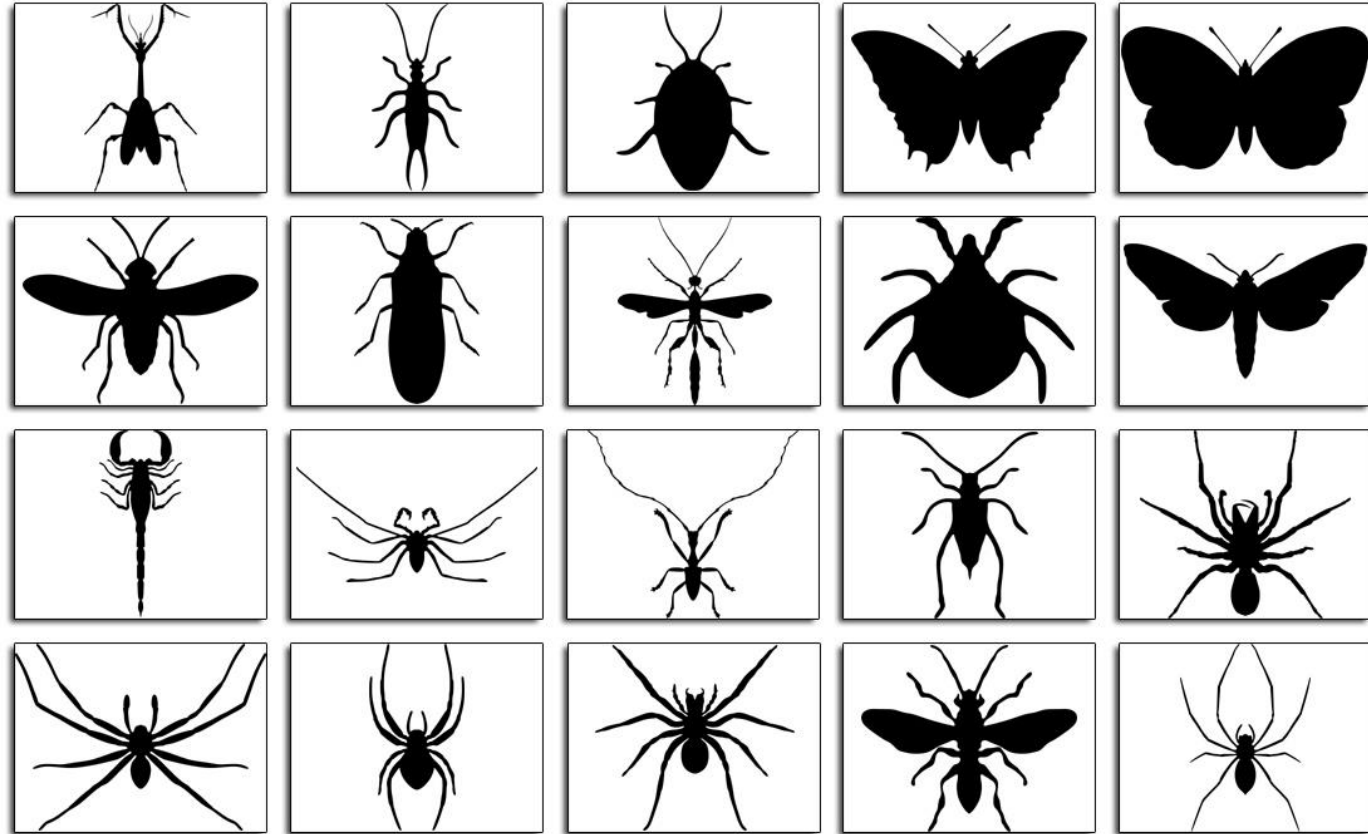


Taxonomy of Hybrid
Quantum-Classical Bugs



Debugging
Quantum Systems

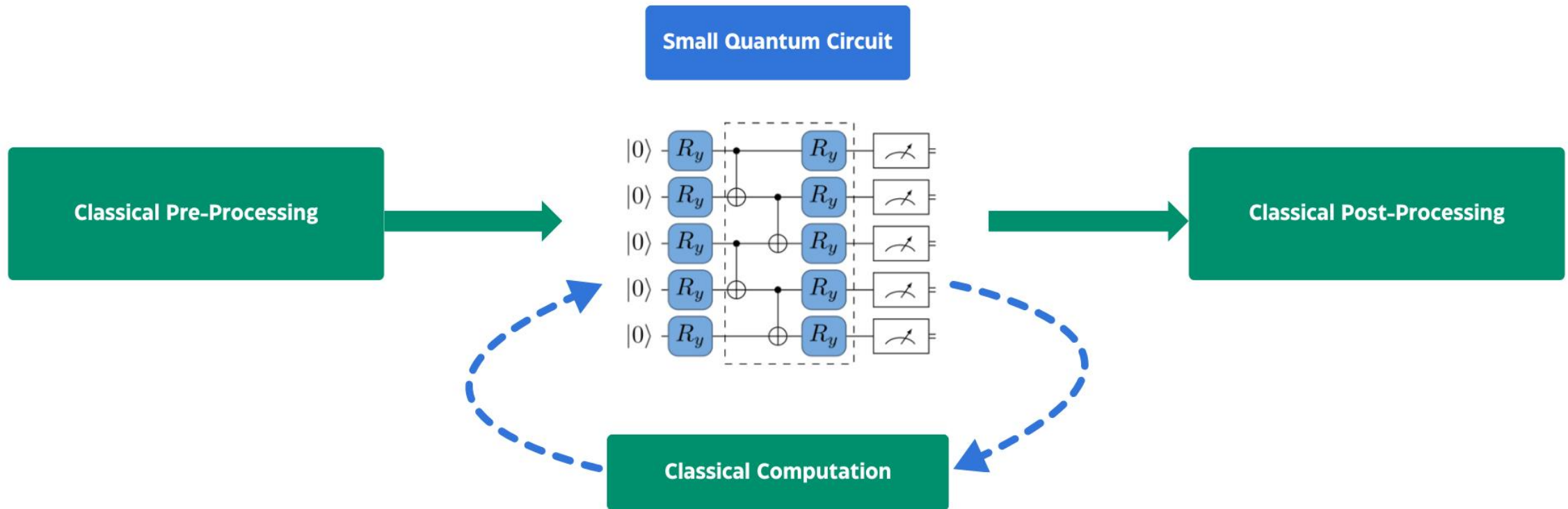
Taxonomy of Real Hybrid Quantum-Classical Bugs



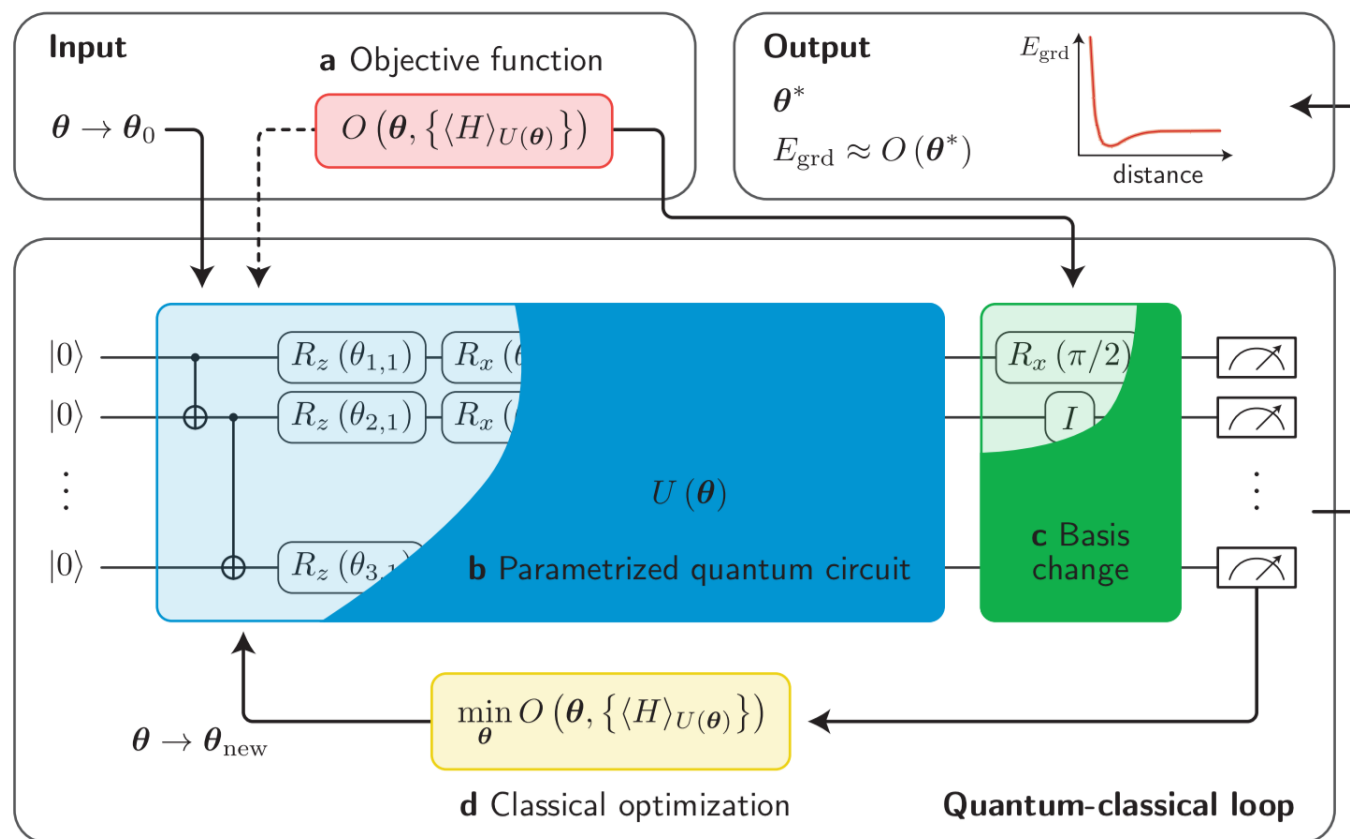
THE SUPER

Based on joint work with: Avner Bensoussan and Gunel Jahangirova

Hybrid Quantum – Classical Algorithms



Example: Variational Quantum Algorithm (VQA)

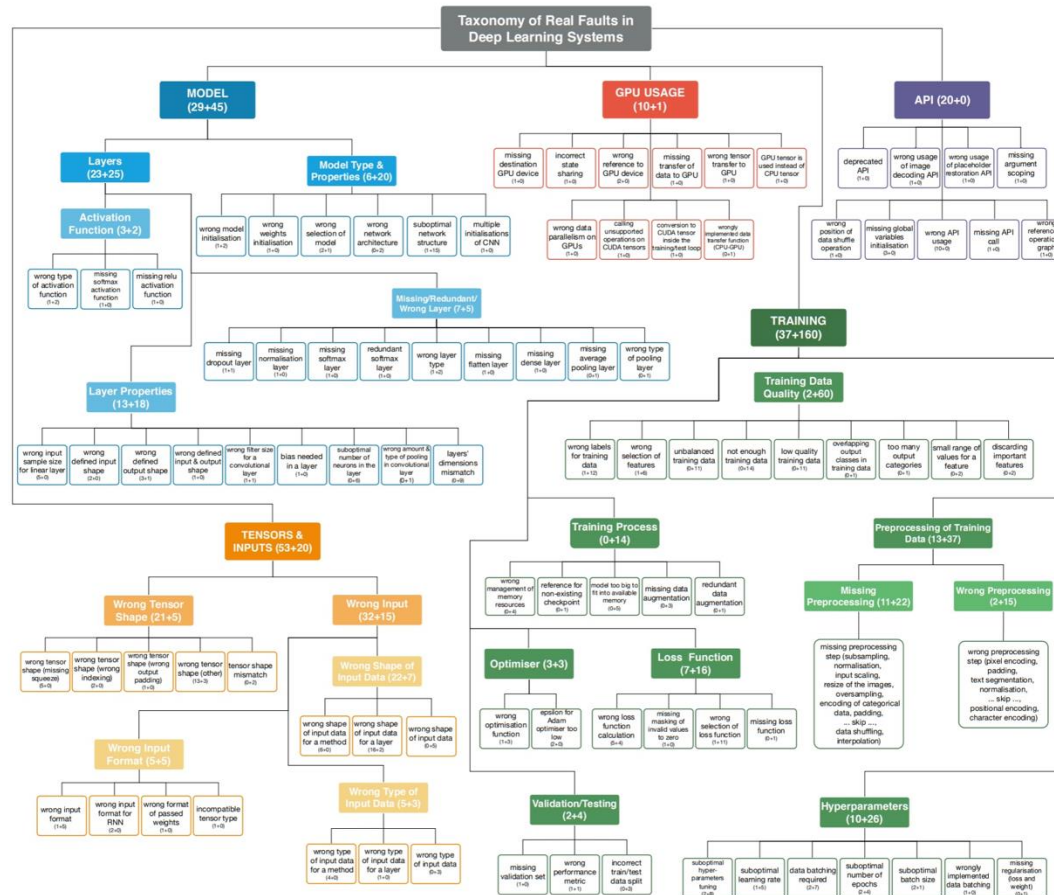


Noisy intermediate-scale quantum (NISQ) algorithms

Kishor Bharti,^{1,*} Alba Cervera-Lierta,^{2,3,*} Thi Ha Kyaw,^{2,3,*} Tobias Haug,⁴ Sumner Alperin-Lea,³ Abhinav Anand,³ Matthias Degroote,^{2,3,5} Hermann Heimonen,¹ Jakob S. Kottmann,^{2,3} Tim Menke,^{6,7,8} Wai-Keong Mok,¹ Sukin Sim,⁹ Leong-Chuan Kwek,^{1,10,11,†} and Alán Aspuru-Guzik^{2,3,12,13,‡}

¹Centre for Quantum Technologies, National University of Singapore 117543, Singapore
²Department of Computer Science, University of Toronto, Toronto, Ontario M5S 2E4, Canada
³Chemical Physics Theory Group, Department of Chemistry, University of Toronto, Toronto, Ontario M5G 1Z8, Canada
⁴QOLS, Blackett Laboratory, Imperial College London SW7 2AZ, UK
⁵current address: Boehringer Ingelheim, Amsterdam, Netherlands
⁶Department of Physics, Harvard University, Cambridge, MA 02138, USA
⁷Research Laboratory of Electronics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA
⁸Department of Physics, Massachusetts Institute of Technology, Cambridge, MA 02139, USA
⁹Department of Chemistry and Chemical Biology, Harvard University, Cambridge, MA 02138, USA
¹⁰MajuLab, CNRS-UNS-NUS-NTU International Joint Research Unit UMI 3654, Singapore
¹¹National Institute of Education and Institute of Advanced Studies, Nanyang Technological University 637616, Singapore
¹²Vector Institute for Artificial Intelligence, Toronto, Ontario M5S 1M1, Canada
¹³Canadian Institute for Advanced Research, Toronto, Ontario M5G 1Z8, Canada

Taxonomy of real bugs, Inspiration



Taxonomy of Real Faults in Deep Learning Systems

Nargiz Humbatova
nargiz.humbatova@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

Gunel Jahangirova
gunel.jahangirova@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

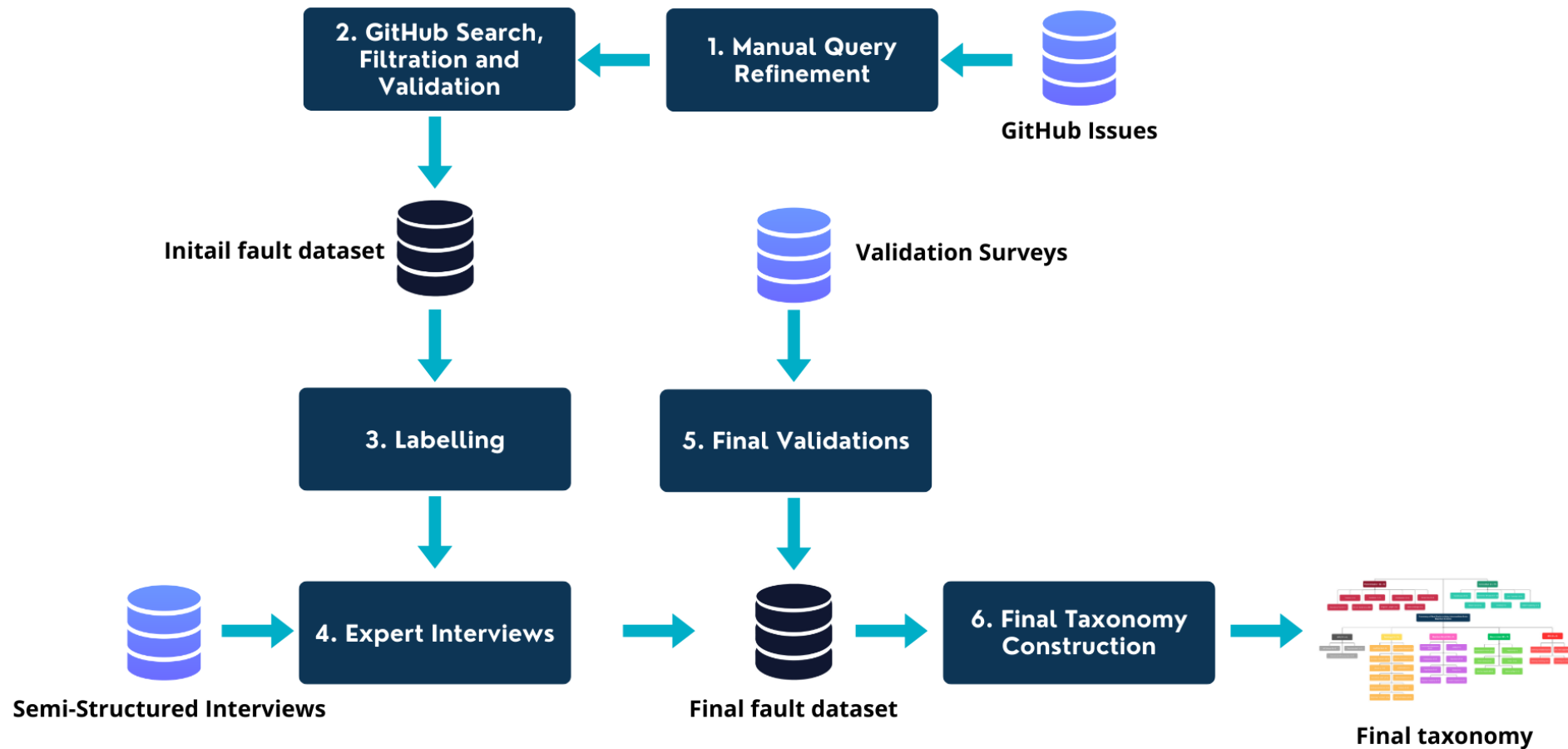
Gabriele Bavota
gabriele.bavota@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

Vincenzo Riccio
vincenzo.riccio@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

Andrea Stocco
andrea.stocco@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

Paolo Tonella
paolo.tonella@usi.ch
Università della Svizzera Italiana (USI)
Lugano, Switzerland

Methodology



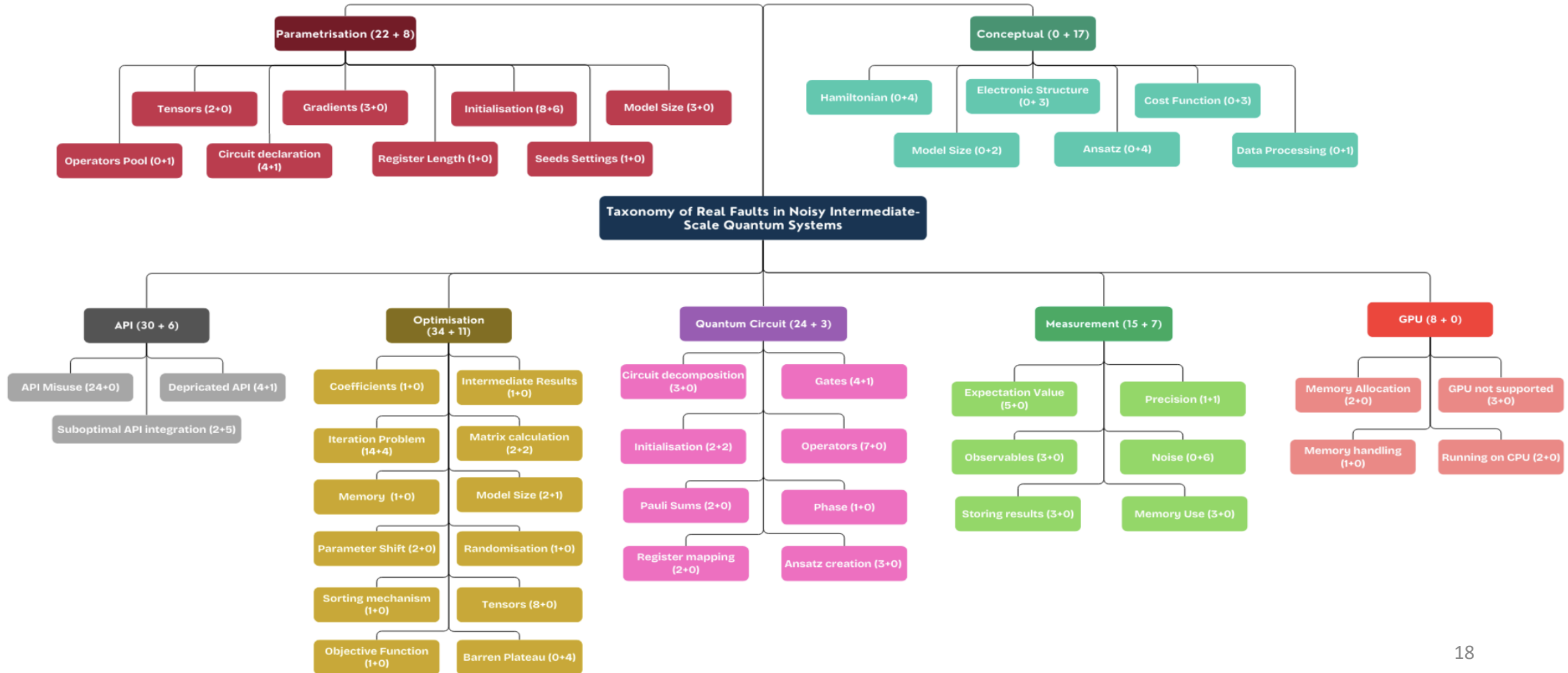
Taxonomy of real bugs, Queries

Query	Number of results	First Filter	Conflicts	Relevant Bugs
VQE	792	162		28
VQA	1886	0	0	0
Variational Quantum eigensolvers	4	0	0	0
Variational Quantum algorithm	100	10	1	1
Quantum Annealing	83	4	0	0
Gaussian boson sampling	8	0	0	0
Analog quantum simulation	26	0	0	0
Digital-analog quantum simulation and	9	0	0	0
Iterative quantum assisted eigensolver	1	0	0	0
Quantum Approximative Optimisation	0	0	0	0
QAOA	466	87	10	27
Quantum Machine Learning	1038	164		20
tfq	226	36	11	19
tensorflow quantum	433	66	22	22
Pilot_study				16
TOTAL	5072	529	44	133

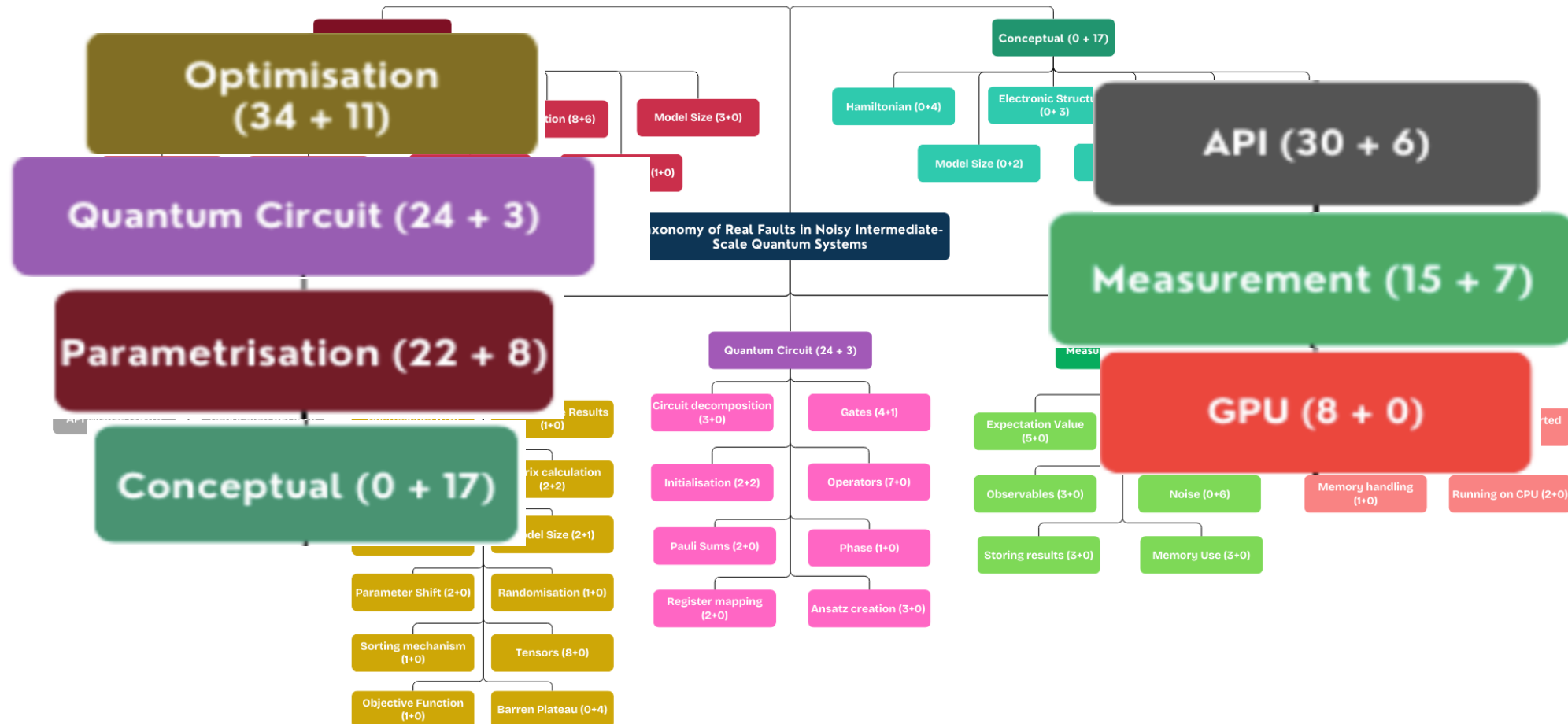
Taxonomy of real bugs, Labels

Symptoms	VQE Levels	Labels
Attribute Error	Ansatz {API}	API Misuse
Circuit Error	Ansatz {Input dimension}	Circuit decomposition does not scale up to 3 qubit gates
DAGCircuit Error	Ansatz {Phase declaration}	Deprecated API
Diag Gates Undefined Error	Ansatz {Structure}	Deprecated API
Dimensionality Issue	GPU {Memory}	Generated binary incompatible with gpu
Error	GPU integration {Memory Allocation}	Global seed is not set
Exception	GPU integration {Not Supported}	Inconstant types for imported Hamiltonians
Extension Error	GPU integration {Running CPU}	Incorrect handling of parameter shifts
Failure	Measurement {API}	Incorrect handling of observables
Index Error	Measurement {Expectation Value}	Incorrect handling of unusual types by register mapping
IndexError	Measurement {Memory Use}	Missing hadamard wall in ansatz
Insufficient Memory Error	Measurement {Observables}	Old API version used
Invalid Argument Error	Measurement {Precision}	Redundant terms in circuit
Key Error	Measurement {Storing results}	Sorting mechanism and probabilities associated wrongly connected
Memory Error	Parametrisation {API}	Suboptimal 2 qubit reduction
Non Differentiable Error	Parametrisation {Circuit declaration}	Suboptimal API integration
NoneType Error	Parametrisation {Gradients}	Suboptimal approximation of the qubit
Operator Not Allowed In Graph Error	Parametrisation {Initialisation}	Suboptimal batch method
Operator Not Allowed In Graph Error	Parametrisation {Model size}	Suboptimal batch size
Out of Memory Error	Parametrisation {Register length}	Suboptimal circuit sampling
Overflow Error	Parametrisation {Seeds settings}	Suboptimal depth size for the model
Performance Loss	Parametrisation {Tensors}	Suboptimal diagonalization method for Double Excitation generator
Premature stopping (when seed set)	Parametrisation {Weights}	Suboptimal gate decomposition
Qiskit Error	Quantum circuit {Circuit decomposition}	Suboptimal gate size
QiskitError	Quantum circuit {Gates}	Suboptimal GPU integration
RAM use too important	Quantum circuit {Initialisation}	Suboptimal GPU memory allocation
Recursion Error	Quantum Circuit {Operators}	Suboptimal handling of PauliSum Operator
Runtime and Type Error	Quantum circuit {Pauli Sums}	Suboptimal handling of redundant qubits
RunTime Error	Quantum Circuit {Phase}	Suboptimal handling of tensors metrics
Segmentation Fault	Quantum circuit {Register mapping}	Suboptimal handling of PauliSum Operator
Slow Performance	Training {Coefficients}	Suboptimal initial point
Tracer Array Conversion Error	Training {Intermediate results}	Suboptimal initialisation of gradients
Type Error	Training {Iteration Problem}	Suboptimal kernel building
Unbounded Local Error	Training {Matrix calculation}	Suboptimal measurement process
Value Error	Training {Memory Leak}	Suboptimal model size
Warning	Training {Model Size}	suboptimal number of repetitions
Wrong output	Training {Objective function}	Suboptimal number of shots
Wrong output (operator type)	Training {Parameter Shift}	Suboptimal observables grouping strategy
Wrong output && RunTime Error	Training {Randomisation}	Suboptimal operation conversion
	Training {Sorting mechanism}	Suboptimal qubit reduction
	Training {Tensors}	Suboptimal rounding procedure

Final Taxonomy



Zooming In

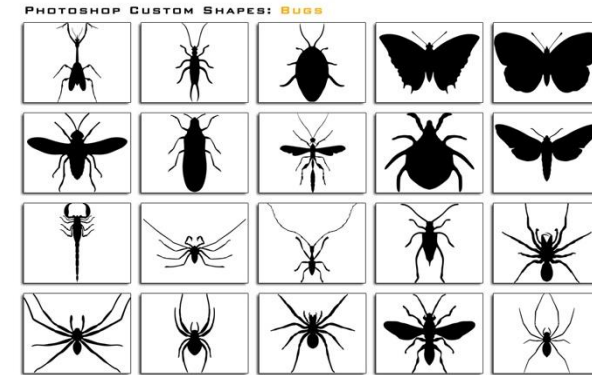




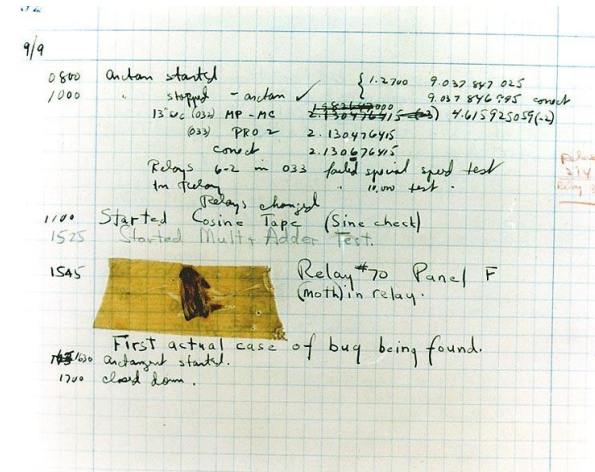
King's
Quantum ✓



Testing
Quantum Systems



Taxonomy of Hybrid
Quantum-Classical Bugs ✓



Debugging
Quantum Systems



Property-Based Testing of Noisy Quantum Programs

Based on joint work with: Shaukat Ali (Simula), Gabriel Pontolillo [King's], Shahin Honarvar (Imperial), Asmar Muqeet (Simula), and Raja Nagarajan (Quentangle)

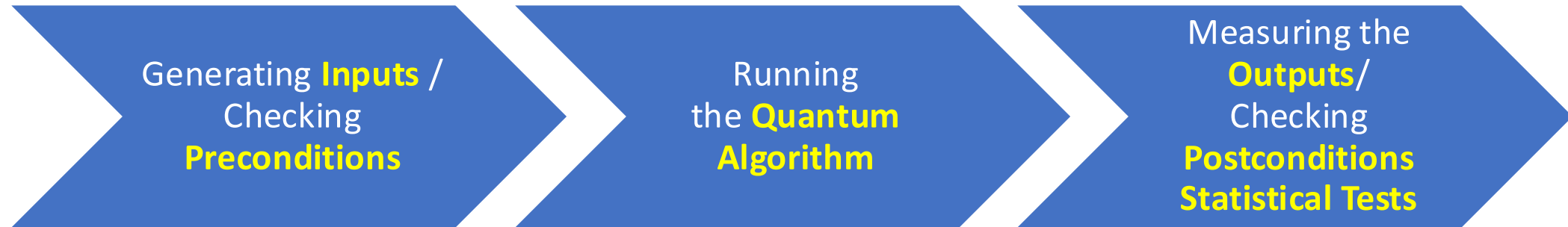
Property-based Testing



*Arts, Hughes, Ulf Norell, Hans Svensson:
Testing AUTOSAR software with QuickCheck. ICST Workshops 2015*

*Hughes, Pierce, Arts, Norell:
Mysteries of DropBox: Property-Based Testing of a Distributed Synchronization Service. ICST 2016*

Property-based Testing (for Quantum Systems)



Properties

Preamble

Precondition

Execution

Assertion /
Post-condition

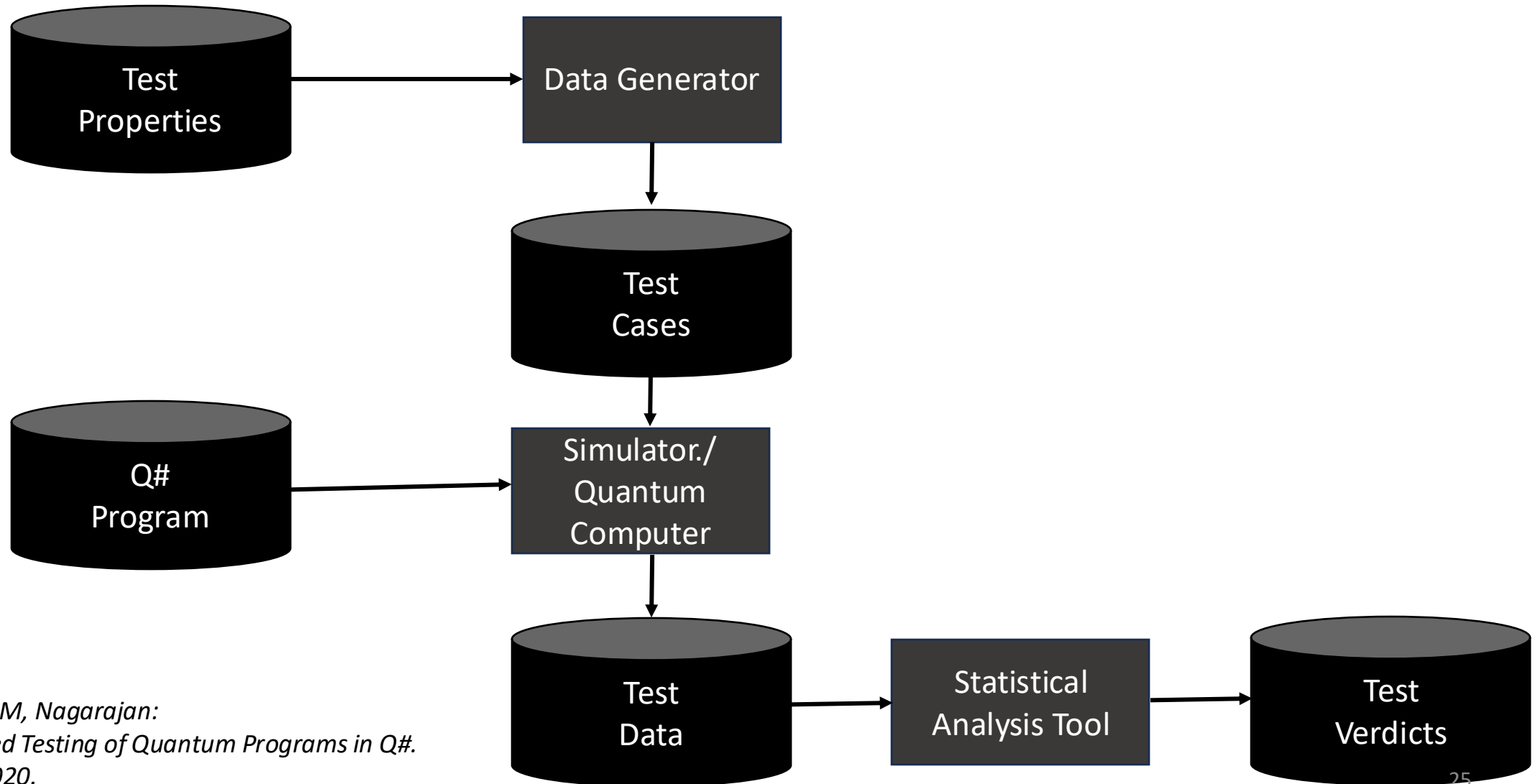
```
//Preamble
Property name;
(numberOfTestCases, confidenceLevel,
  numberOfMeasurements, trials);

//Precondition and initialisation
{ $q_0 \dots q_n$  : Qubit ( $\theta$  interval)( $\phi$  interval)};
{ $b_0 \dots b_n$  : Bool};

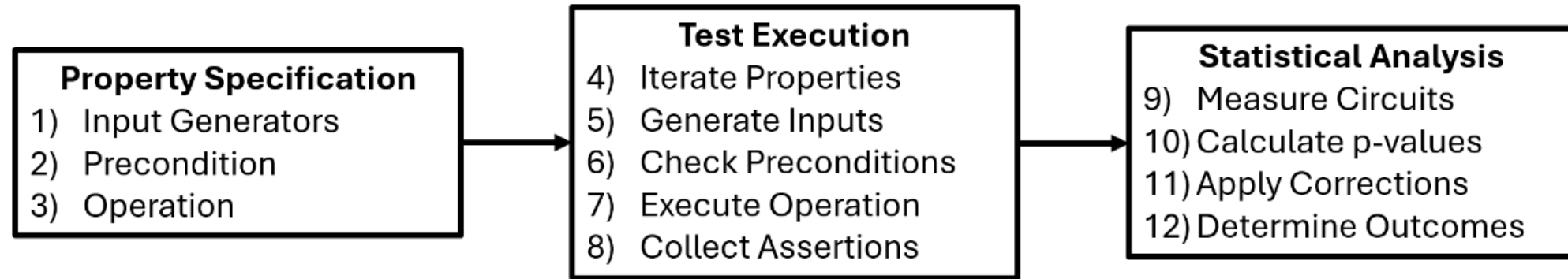
//Invoking system under test
operation_name (arguments) : return type;

//Postcondition
//Any combination of the following assertions
[AssertProbability( $q_0$ , 0, proposed propability)];
[AssertEntangled( $q_0, q_1$ )];
[AssertEqual( $q_0, q_1$ )];
[AssertTeleported( $q_0$ )];
[AssertTransformed( $q_0$ , ( $\theta$  interval)( $\phi$  interval))];
```

Initial Tool Architecture

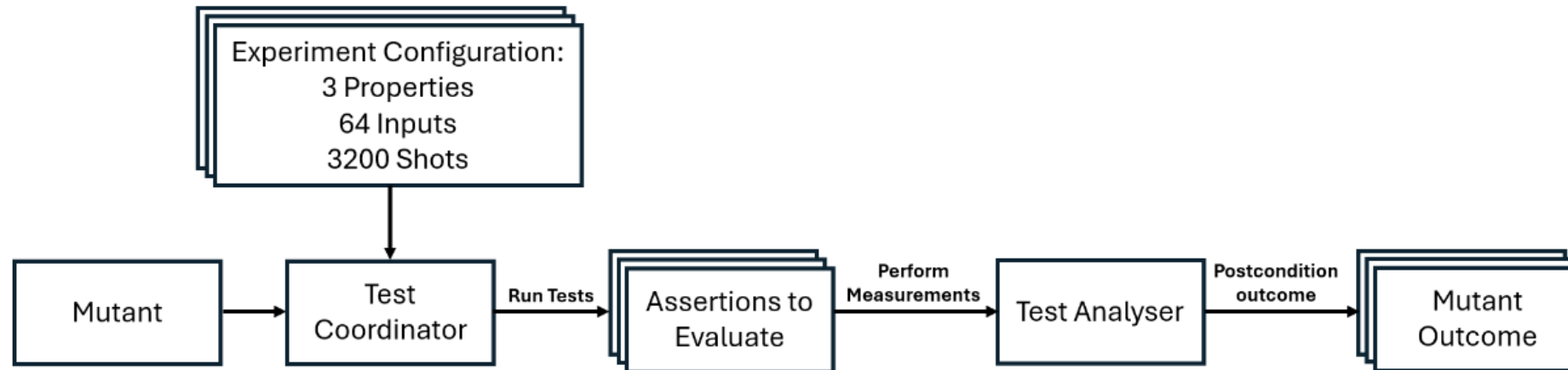


QuCheck



Pontolillo, Grzesiuk, MRM: QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit. Arxiv, Figshare 2025.

QuCheck: Evaluation



Pontolillo, Grzesiuk, MRM: QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit. Arxiv, Figshare 2025.

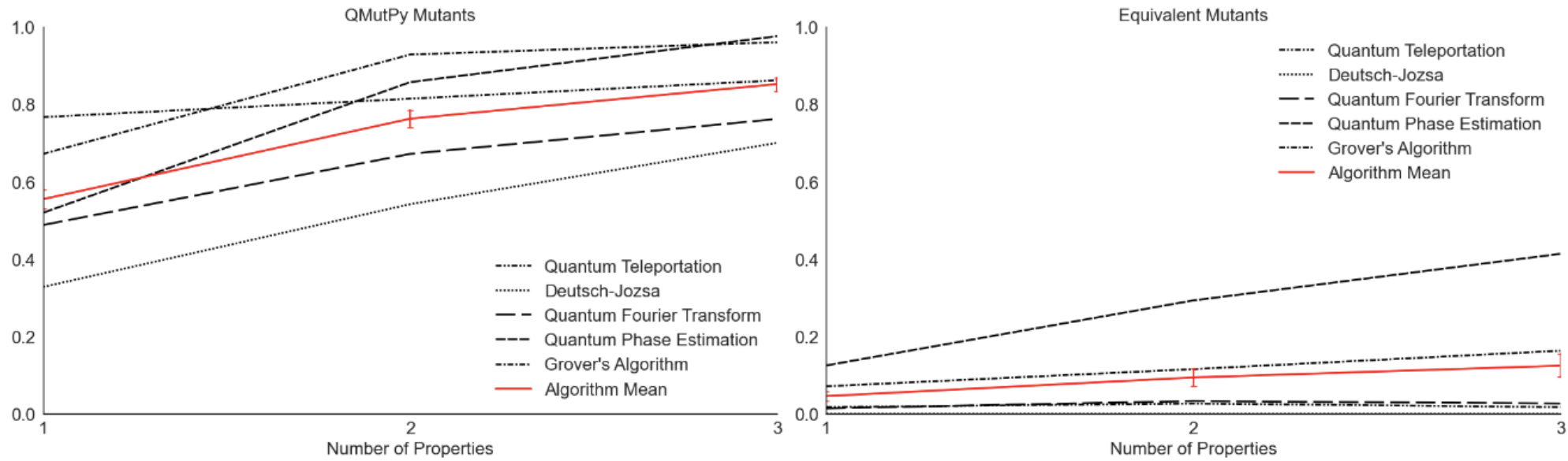
Evaluation: Mutation Analysis

- Statement deletion: $Z(q); \rightarrow -$
- Statement duplication: $Z(q); \rightarrow Z(q); Z(q);$
- Replacement of Boolean relations: $M(q) == \text{One} \rightarrow M(q) != \text{One}$
- Replacement of q-operators: $Z(q); \rightarrow X(q);$
- Changing the order of q-operators: $H(--); \text{CNOT}(...); \rightarrow \text{CNOT}(...); H(--);$
- Replacement of a variable with another: $H(q_0); \rightarrow H(q_1);$

Mendiluze, Ali, Arcaini, Yue: Muskit: A Mutation Analysis Tool for Quantum Software Testing. ASE 2021.

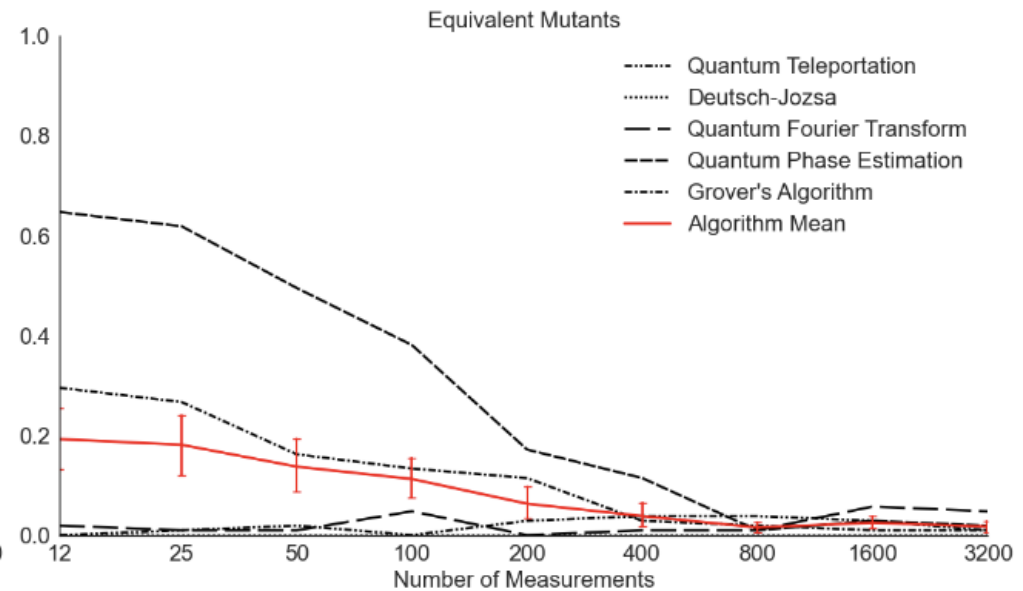
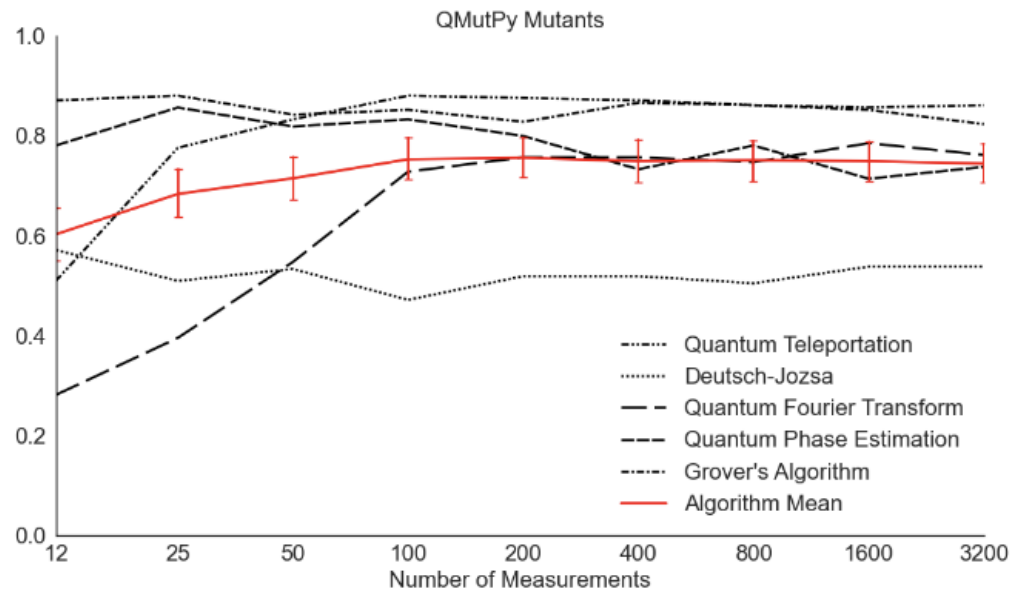
Fortunato, Campos, Abreu: QMutPy: a mutation testing tool for Quantum algorithms and applications in Qiskit. ISSSTA 2022.

Evaluation: Effect of Properties

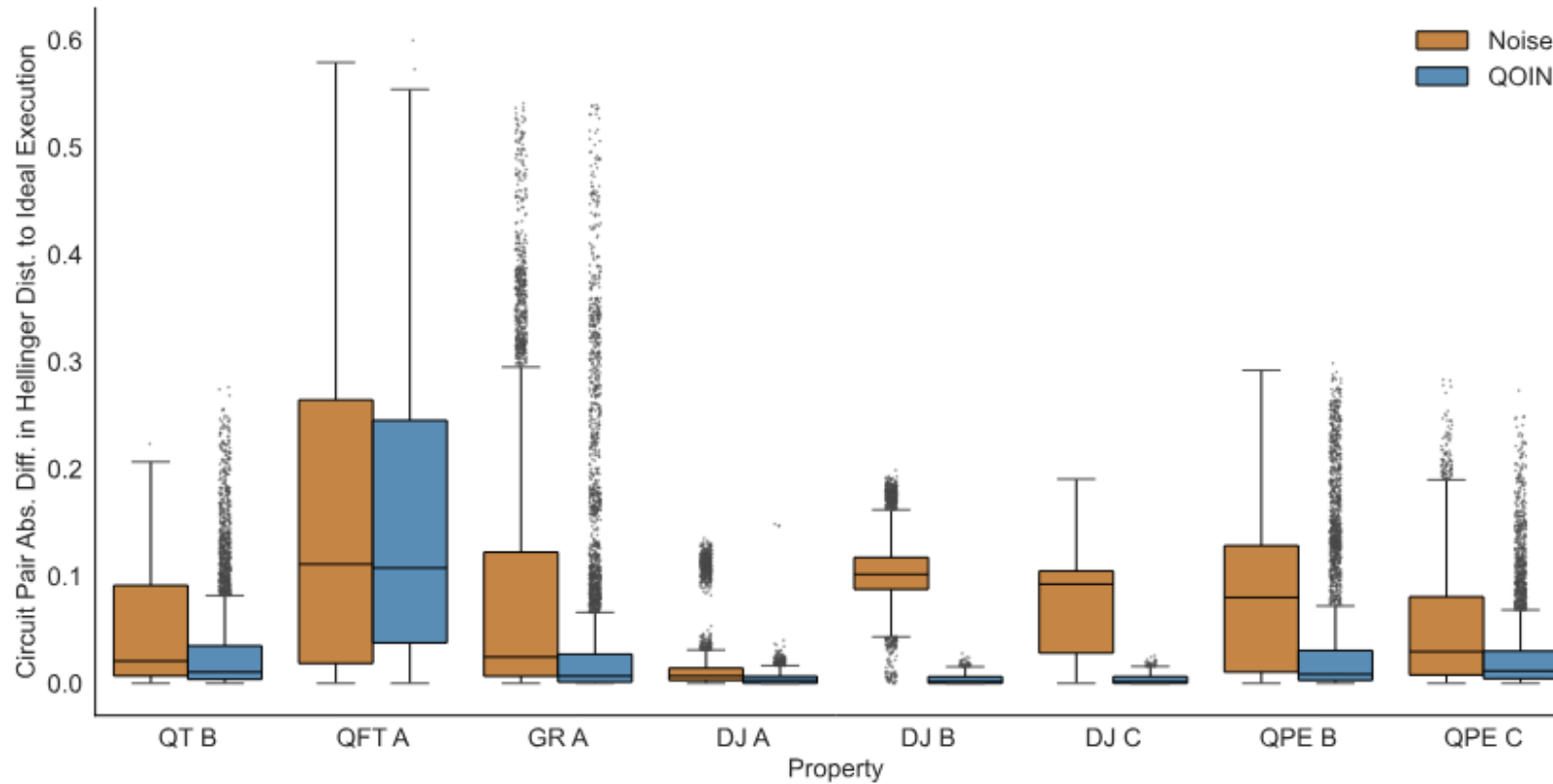


Pontolillo, Grzesiuk, MRM: QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit. Arxiv, Figshare 2025.

Evaluation: Effect of Shot#



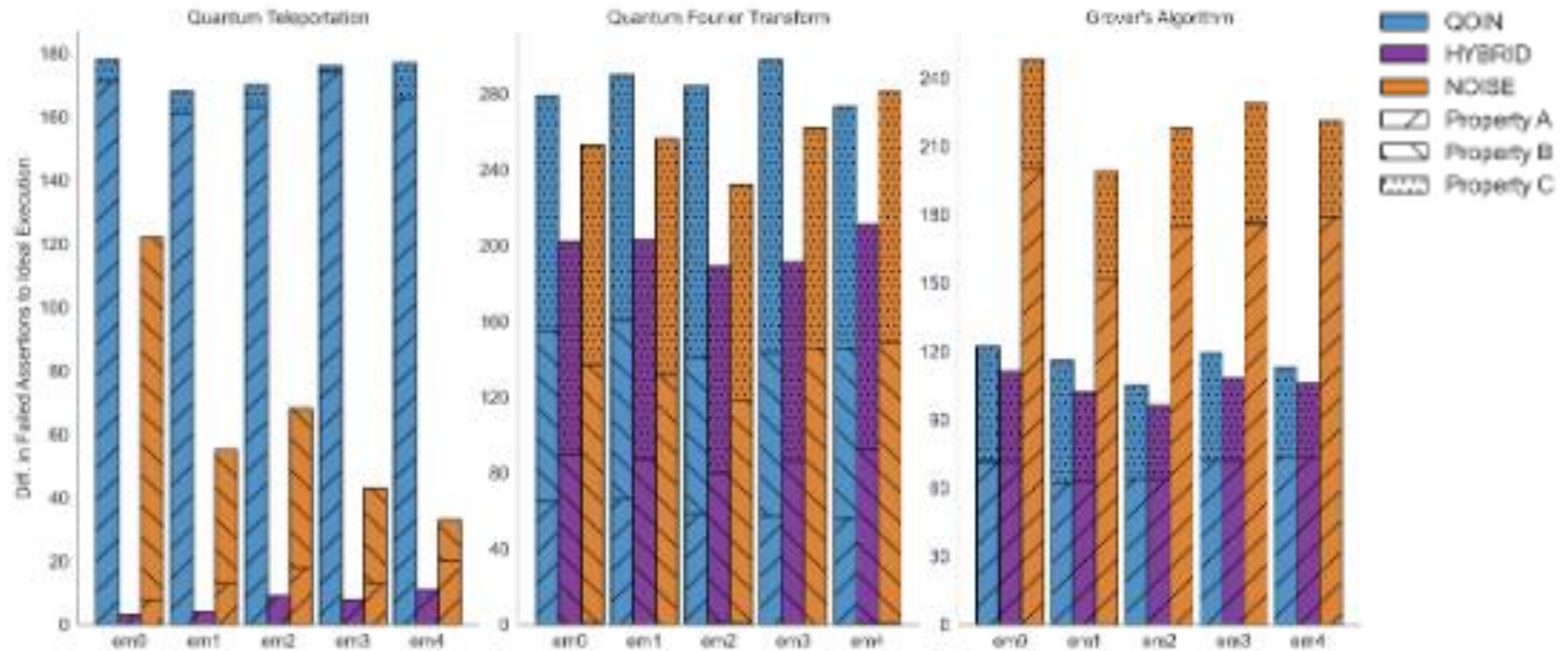
Mitigating Noise



Muqheet, Yue, Ali, and Arcaini, Mitigating noise in quantum software testing using machine learning, IEEE TSE 2024.

Pontolillo, Muqheet, Ali, MRM. From Ideal to Noisy: Adapting Property-Based Testing for Real-World Noisy Quantum Computers. QCE 2025

Mitigating Noise: Overcoming Overfitting



Pontolillo, Muqet, Ali, MRM. From Ideal to Noisy: Adapting Property-Based Testing for Real-World Noisy Quantum Computers. QCE 2025

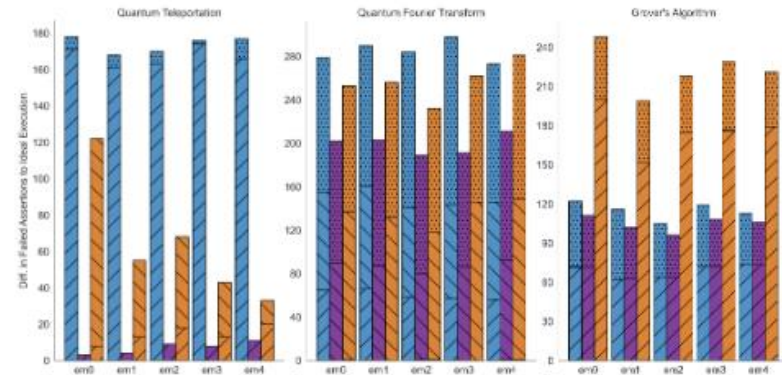
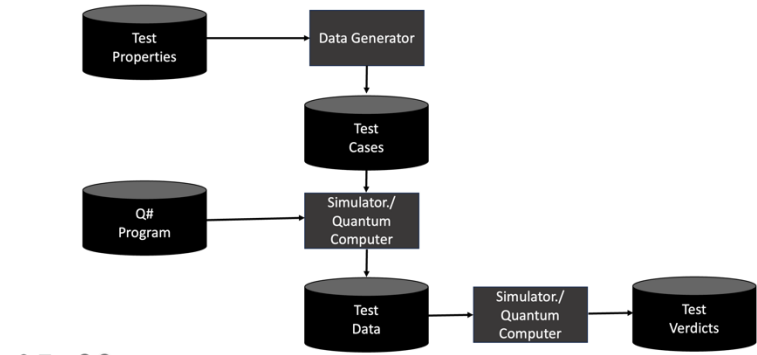
Summary



Teleportation:
16/20 mutants killed



Superdense coding:
12/20 mutants killed



Honarvar, MRM, Nagarajan: Property-based Testing of Quantum Programs in Q#. Q-SE@ICSE 2020.

Pontolillo, MRM: A Multi-Lingual Benchmark for Property-Based Testing of Quantum Programs. Q-SE@ICSE 2022.

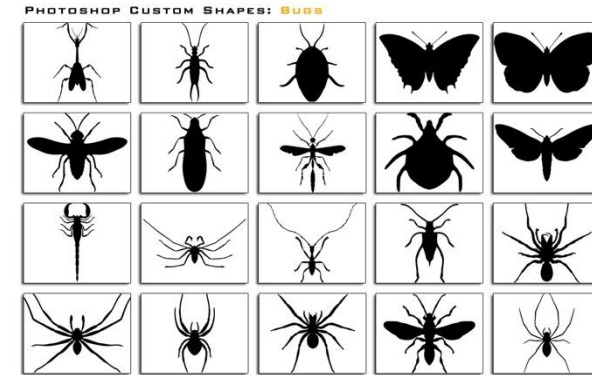
Pontolillo, Grzesiuk, MRM: QuCheck: A Property-based Testing Framework for Quantum Programs in Qiskit. Arxiv 2025.



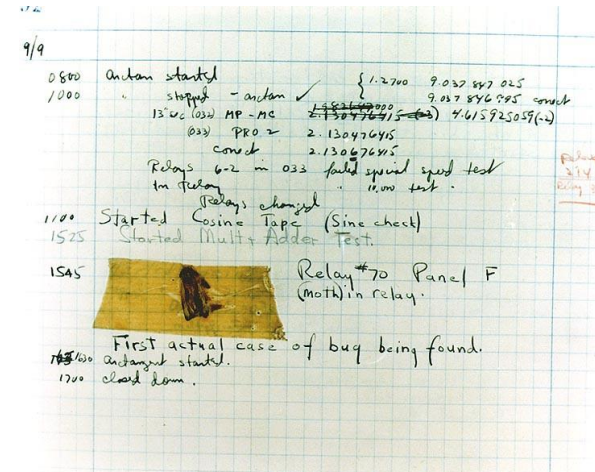
King's
Quantum ✓



Testing
Quantum Systems ✓



Taxonomy of Hybrid
Quantum-Classical Bugs ✓



Debugging
Quantum Systems



Property-Based Automated Debugging

Joint work with Gabriel Pontolillo

We extend **delta debugging** by applying **property-based test oracles** and evaluate its *effectiveness* at **isolating faults** within quantum program updates.

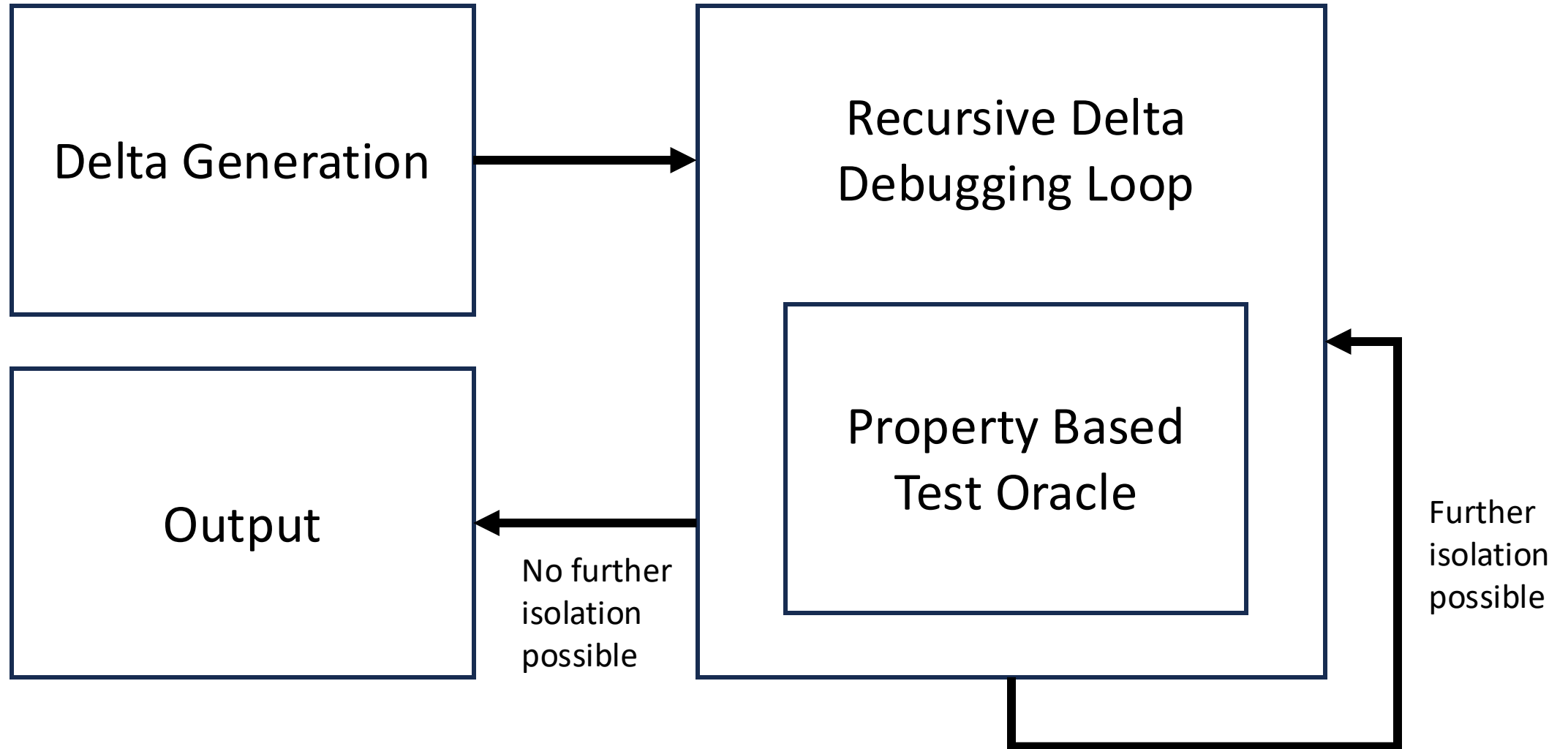
Debugging: Challenges

- Oracle problem: complex logic of (“simple”) quantum algorithms
- Inspecting intermediate states
- Infinite state space of a qubit
- Superposition and entanglements

Debugging: Our Setting

- A **passing** and a **failing** circuit (regression testing)
- Goal: **identify the fault** causing the failure, leaving out the **chaff**

Debugging: Our Setting



Delta Debugging

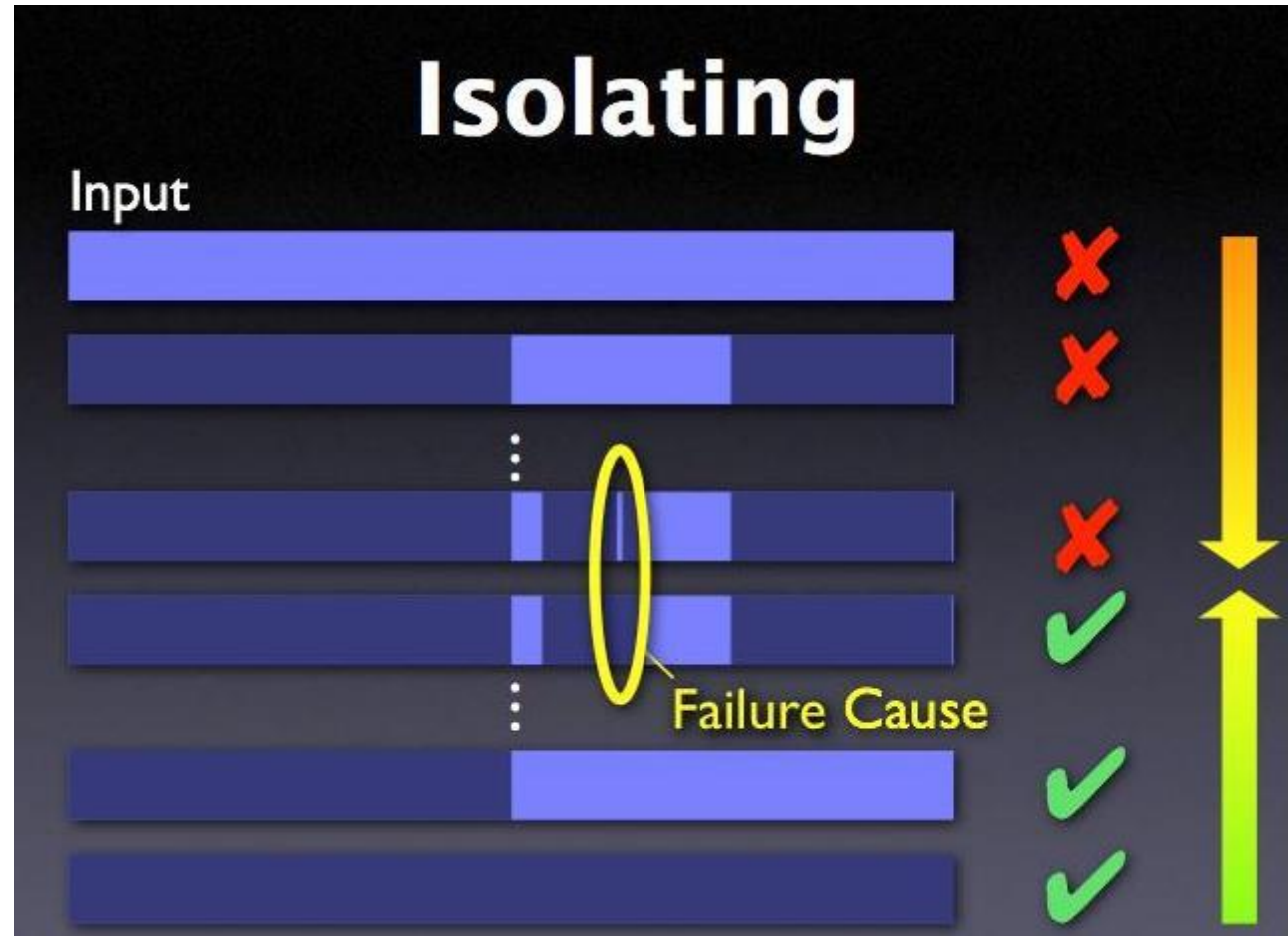
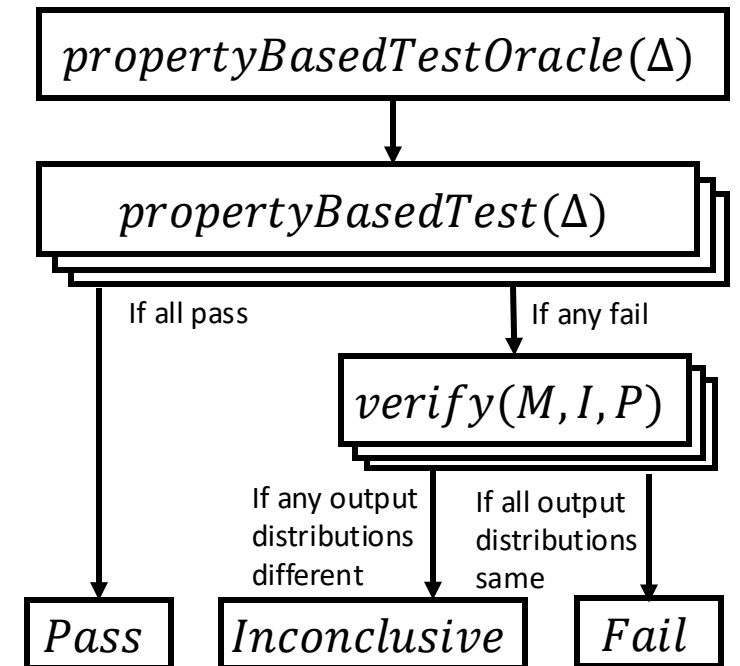


Illustration due to Andreas Zeller

We extend **delta debugging** by applying **property-based test oracles** and evaluate its *effectiveness* at **isolating faults** within quantum program updates.

Property-Based Oracles

- Multiple property-based tests executed
- When a failure is observed:
 - We **verify that the failure is the same as the initial failure**.
- The initial circuit is tested with the **same configuration** that caused the intermediate circuit failure.



We extend **delta debugging** by applying **property-based test oracles** and evaluate its *effectiveness* at **isolating faults** within quantum program updates.

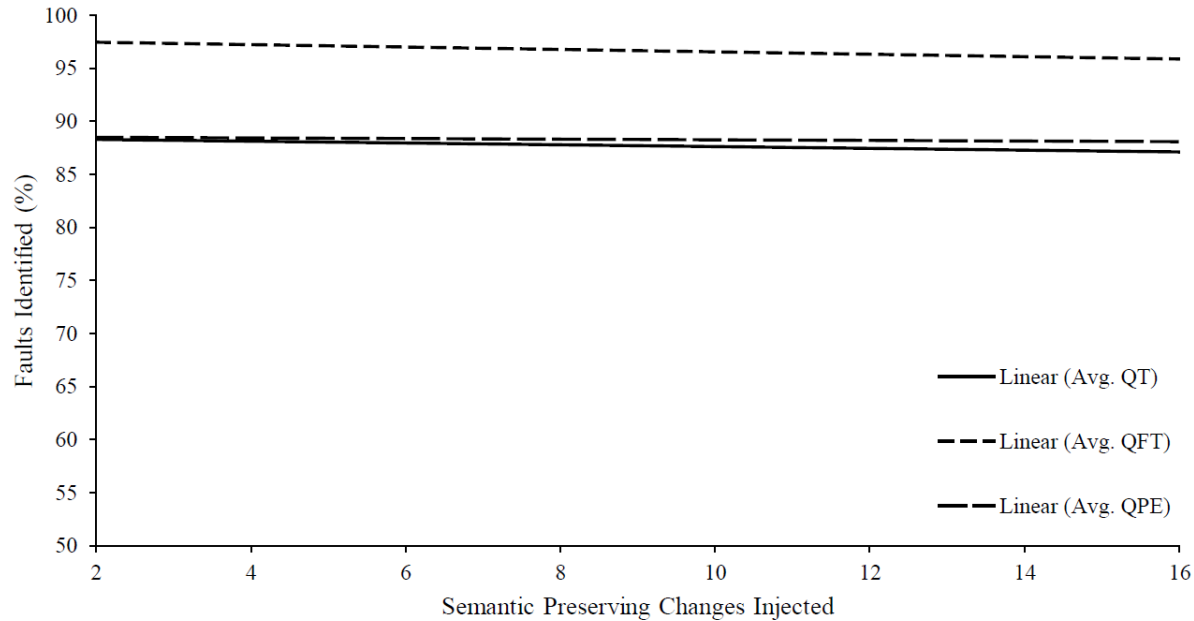
Effectiveness

- **RQ1:** Is delta debugging **effective** at locating faults in quantum program updates?
- **RQ2:** Does the effectiveness of the technique correlate with the **number of properties** evaluated within the property-based test oracle?
- **RQ3:** Does the effectiveness of the technique increase with the **number of inputs** generated per property-based test?

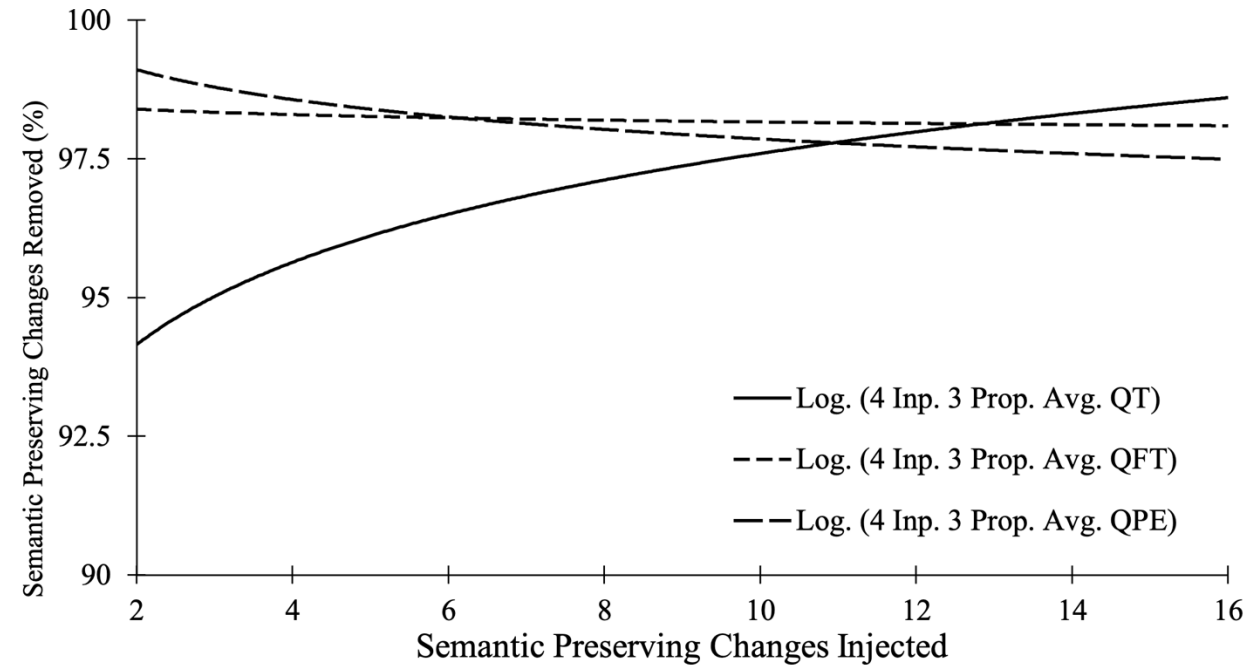
Effectiveness

- **Accuracy:** % of runs the failure-causing fault included in output
- **Precision:** % of chaff excluded from output in each run

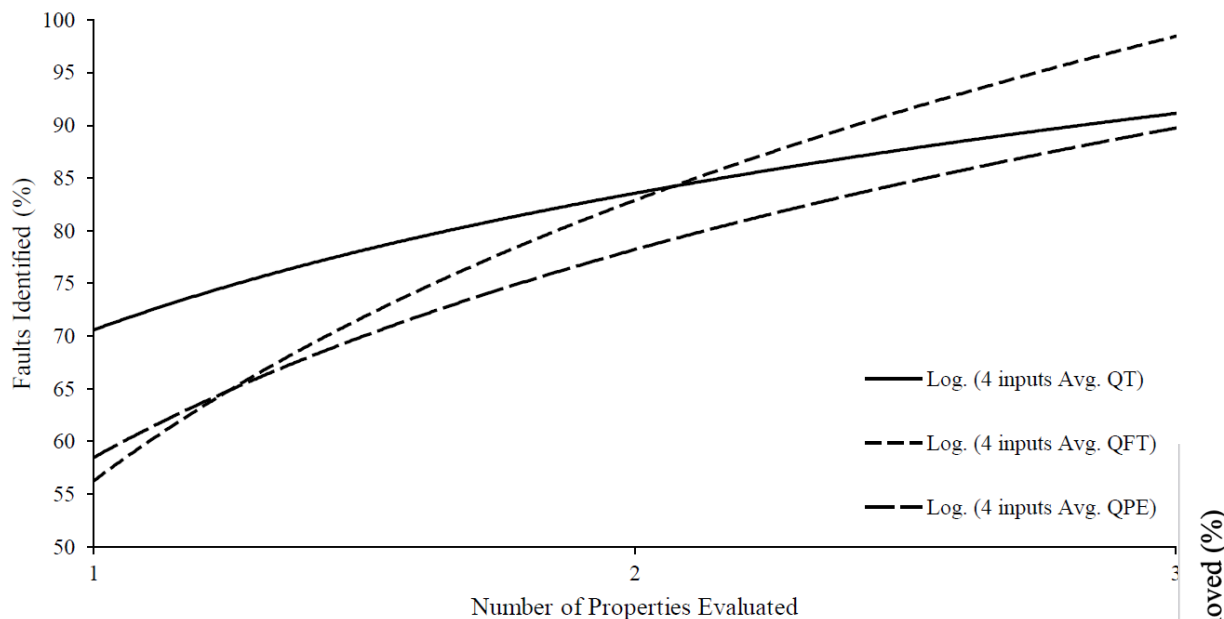
Effectiveness: Increasing Chaff



[G. Pontolillo and MRM.
Delta Debugging for Property-Based Regression Testing of
Quantum Programs. Q-SE 2024 @ ICSE 2024]

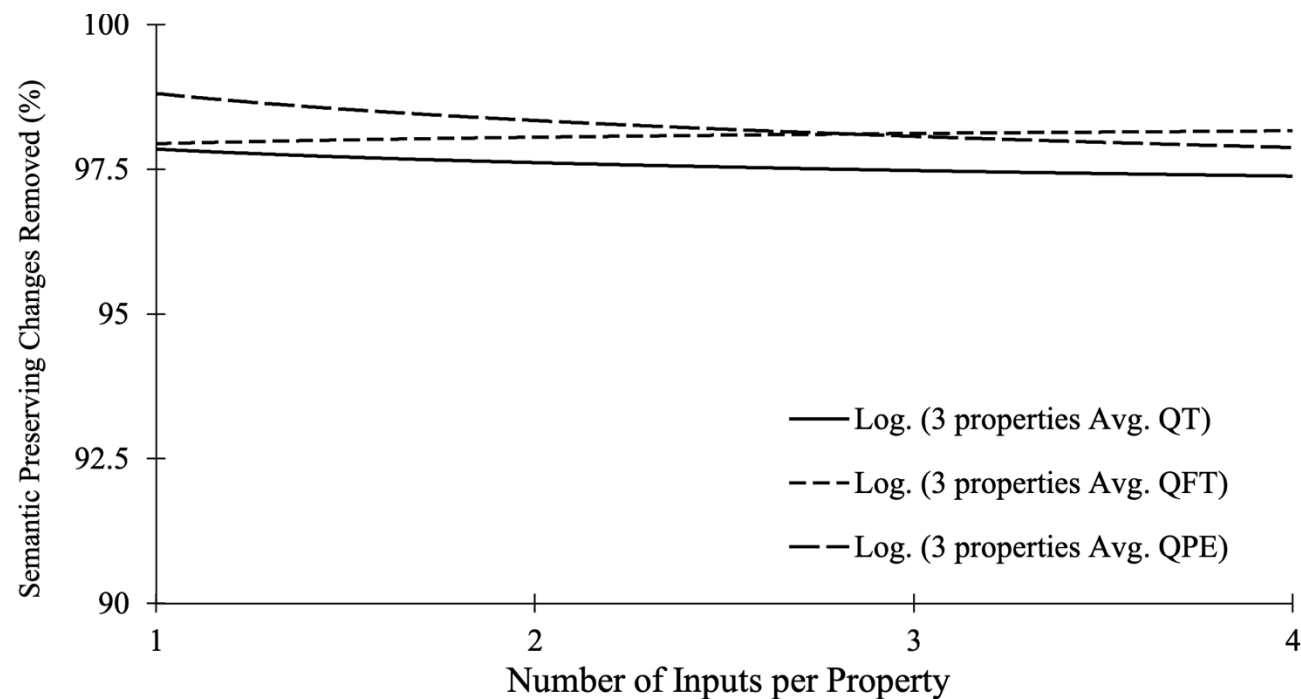


Effectiveness: Increasing Properties

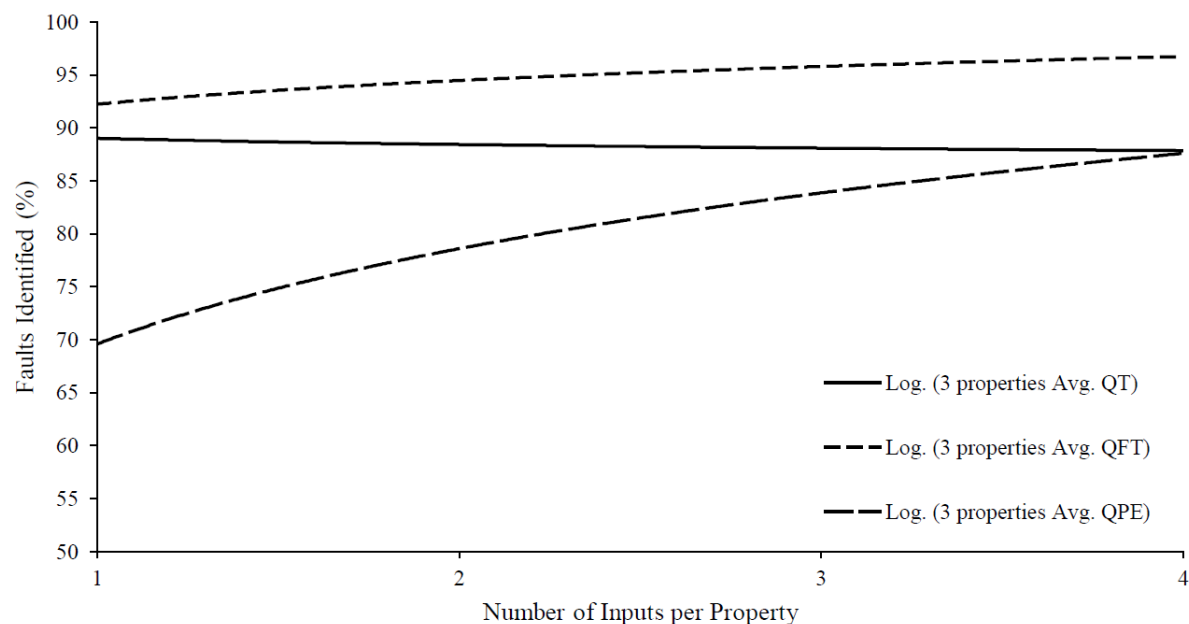


[G. Pontolillo and MRM.

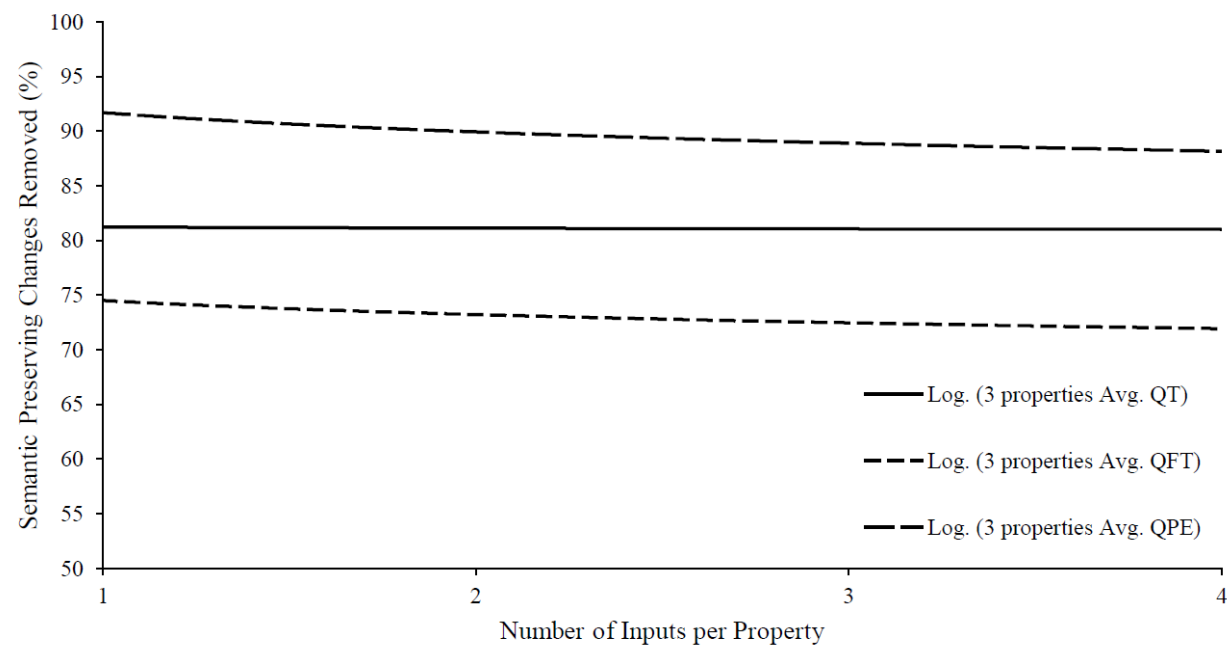
Delta Debugging for Property-Based Regression Testing of Quantum Programs. Q-SE 2024 @ ICSE 2024]



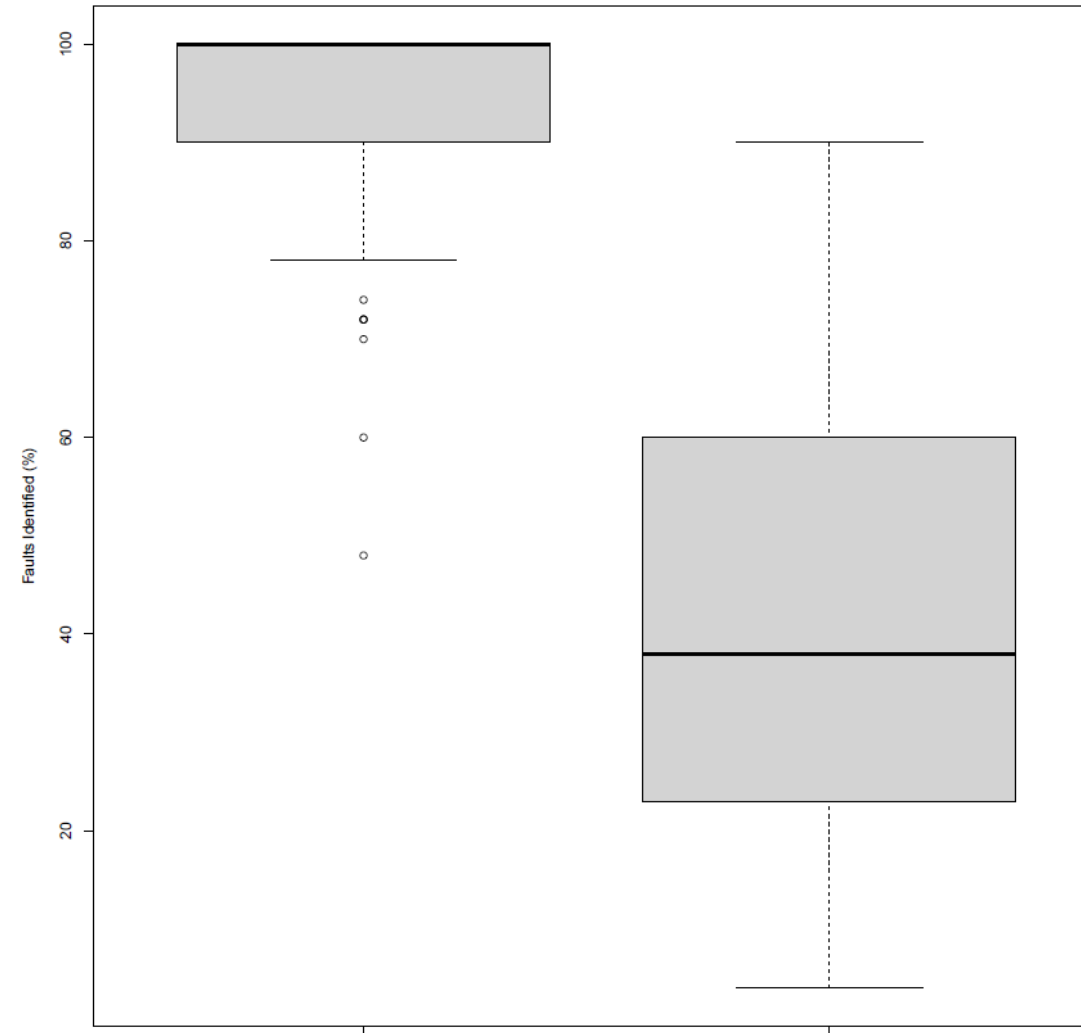
Effectiveness: Increasing Inputs



[G. Pontolillo and MRM.
Delta Debugging for Property-Based Regression Testing of
Quantum Programs. Q-SE 2024 @ ICSE 2024]



Effectiveness: Localising Failures



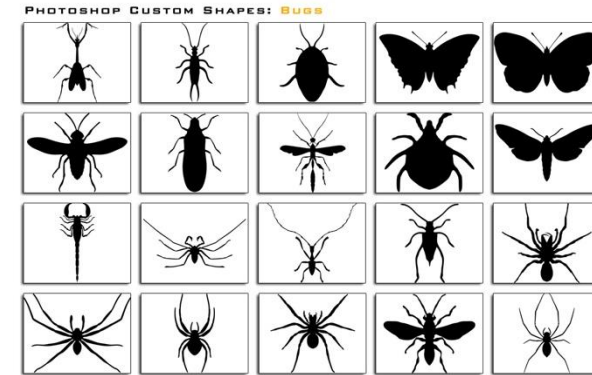
We extend **delta debugging** by applying **property-based test oracles** and evaluate its *effectiveness* at **isolating faults** within quantum program updates.



King's
Quantum ✓



Testing
Quantum Systems ✓



Taxonomy of Hybrid
Quantum-Classical Bugs ✓



Debugging
Quantum Systems

Ongoing work

- **Noise-aware mutation operators:**
training mutation operators and oracles
to distinguish between faults and noise
- Information-theoretic **foundations of quantum testability**,
defining test-diversity, generating diverse test-cases
- **Run-time verification** of hybrid quantum-classical systems (accepted at Q-SE 2026)
- **Quantum annealing** for testing cyber-physical systems (to appear soon, pre-print on Arxiv)

Thank you very much!

mohammad.mousavi@kcl.ac.uk